



PolarFire Xilinx Tradeoff

Issue 1.0

Table of Contents

1. PolarFire and Xilinx accelerator trade-off	3
2. Introduction	4
• 2.1. Objective	4
• 2.2. Scope	4
• 2.3. Applicable Documents	5
• 2.4. Reference Documents	5
• 2.5. Definitions	5
3. How the network is split	6
4. Embedding a PyTorch model on PolarFire	7
5. Why the QAT checkpoints were used	8
6. Xilinx accuracy baseline	8
• 6.1. Object detection (FCOS)	8
• 6.2. Keypoint regression (MobilePose)	9
7. Latency	10
• 7.1. Xilinx	10
• 7.2. PolarFire	11
• 7.3. Comparison	11
8. Qualitative overlays	12
• 8.1. Keypoint regression	12
• 8.2. Object detection	14
9. Conclusion	16
10. Open items	16

PolarFire Xilinx Tradeoff

1. PolarFire and Xilinx accelerator trade-off

Date created: 05/10/2026

Distribution:

- LMO (internal),
- European Space Agency (ESA)

Change Log:

- 1.0: First release

	Position	Name	Signature	Date
Prepared by	Computer Vision Engineer	Maciej Zurad		

Reviewed by

Approved by

This document is of Luxembourg origin. Copyright © LMO Sarl.

2. Introduction

The DIOSSA RPO payload estimates the 6-degree-of-freedom pose of a target spacecraft from monocular images. The vision chain is two convolutional networks in series: FCOS for object detection, then MobilePose for keypoint regression. In CCN1 those networks were trained in floating point, refined with quantization-aware training, and compiled onto the Xilinx Deep Learning Processing Unit (DPU) in an UltraScale+ MPSoC through Vitis-AI 3.5.

This note compares that baseline with the same networks compiled for the Microchip PolarFire SoC and its VectorBlox accelerator (SDK 3.1). The architectures and the CCN1 weights are unchanged. Under comparison are the accelerator and the compilation path.

2.1. Objective

The objective is to record whether the CCN1 networks compile and run on PolarFire, and to set the measured latency beside the Xilinx DPU and CPU-head measurements from the earlier benchmarking campaign.

Accuracy on the PolarFire test set (IoU, keypoint error) is not part of this issue yet, as described in the Open items section.

2.2. Scope

This note focuses on two specific models and configurations from CCN1:

- FCOS for object detection, using an input size of 384×288 (W×H), named `quare-delf`.
- MobilePose for keypoint regression, using an input size of 224×224 and 20 keypoints, named `bijou-rasp`.

An additional FCOS network at 640×480 resolution was also trained during CCN1 and may be referenced for context, but it was not deployed or compiled for PolarFire as part of this comparison.

2.3. Applicable Documents

AD-n	Document Title	Issue
AD-1	PL23-0005-DJF-0001 i1.1 (DIOSSA-CCN1)	1.1
AD-2	PL24-0001-DJF-0002 i1.1 (Algorithm Design Justification File)	1.1

2.4. Reference Documents

RD-n	Document Title
RD-1	VectorBlox SDK, Tutorial Walkthrough Guide (VectorBlox-SDK/docs/tutorial_walkthrough_guide.md)
RD-2	Microchip VectorBlox Accelerator SDK, product description of the 3.1 release

2.5. Definitions

Acronym	Meaning
COCO	Common Objects in Context dataset
DPU	Xilinx Deep Learning Processing Unit
EDGE	The part of the network compiled onto the accelerator
FLOAT	The float32 network, evaluated on a GPU

Acronym	Meaning
FCOS	Fully Convolutional One-Stage object detector
PTQ	Post-training quantization. Scales and zero-points are computed from a calibration set after training.
QAT	Quantization-aware training. The stored weights are float32, but training was performed under a fake-quantization constraint.
VNNX	VectorBlox binary executed by the accelerator

3. How the network is split

Following the previous DIOSSA phases, each network is divided into two parts.

The accelerator part, referred to below as the edge, runs on the FPGA. It is made as large as the core will accept, so that the convolutional trunk uses the accelerator and the CPU is left with as little work as possible.

The second part stays on the CPU. It is the tail that cannot be placed on the accelerator. For VectorBlox, the “tail” refers to any part of the network that cannot make it through the conversion to a fully integer INT8 TensorFlow Lite graph and subsequent compilation with `vnnx_compile`. The reasons a layer may be left out include:

- the operator is outside the INT8 TensorFlow Lite set that VectorBlox compiles [RD-1];
- the layer is post-processing rather than a convolution. For example: box decoding, non-maximum suppression, top-k selection, and the reduction from a heatmap to a coordinate;
- the tensor shape is dynamic, or an attribute of the operator (rank, dilation, alignment) is not accepted by the compiler;
- the activation memory of the whole graph does not fit the selected core. The build used here is the V1000 configuration with no compression.

On PolarFire the CPU tail is an ONNX graph executed by ONNX Runtime on the RISC-V application cores. Accelerator outputs are dequantized on the CPU and then passed into that graph. FCOS consumes the box, class, and centerness feature maps and emits boxes, scores, and labels.

MobilePose consumes the unnormalized heatmaps and emits coordinates and normalized heatmaps. The same split was used on the Xilinx side: the DPU runs an `.xmodel`, and the post-processing head runs as ONNX on the MPSoC CPU.

4. Embedding a PyTorch model on PolarFire

VectorBlox does not natively support PyTorch models. To deploy a PyTorch model on PolarFire's FPGA accelerator, it must be converted into VNNX format using the VectorBlox SDK.

We use the following steps:

1. Load the PyTorch checkpoint and export two ONNX files. One is the accelerator "edge" part. The other is the CPU "tail" part.
2. Build a calibration sample. One hundred images are taken from the CCN1 test sample, resized to the network input, and stored as a NumPy array. The array is normalized to the 0–1.0 range.
3. Simplify the ONNX graph with `onnxsim`, so the converter receives a static graph without extra nodes or artifacts left from training.
4. Convert ONNX to a fully integer INT8 TensorFlow Lite model with `onnx2tf`. This step both translates the graph and quantizes it. The calibration array is what the quantizer uses to pick a scale and a zero-point per tensor. Mean and standard deviation are used to normalize the calibration array here.
5. Insert preprocessing with `tf_lite_preprocess`. Mean and standard deviation are written into the graph, scaled into the 0–255 pixel range, so the accelerator consumes the image with the same normalization the network was trained with. The tool also inserts the uint8-to-int8 quantize at the input.
6. Compile with `vnnx_compile -s V1000 -c ncomp`. The output is a `.vnnx` binary for the V1000 core, without weight compression.

If anything fails, it is most likely because a layer is not supported by the compiler, in which case the model must be split in a different place.

5. Why the QAT checkpoints were used

The first attempt was to embed the CCN1 float checkpoints (`licit-weal` for FCOS, `blest-harl` for MobilePose). That quantization did not complete. `onnx2tf` could not assign a scale and a zero-point on those weights, so no INT8 TensorFlow Lite file was produced and there was nothing to compile.

The weights that did quantize are the CCN1 quantization-aware checkpoints, `quare-delf` and `bijou-rasp`. They are still stored as float32. They were trained with Vitis-AI fake quantization, so the values already sit on a grid that post-training quantization (PTQ) can represent. With those weights the same calibration sample produces a finite scale and zero-point, and we can proceed with the compilation.

This is the same pair that was embedded on the Xilinx DPU in CCN1. The float parents are the accuracy reference. They are not the binaries on either accelerator.

6. Xilinx accuracy baseline

The figures in this section are copied from AD-1. They were measured on the CCN1 test set. They are not re-measured in this issue, and they are not PolarFire numbers.

6.1. Object detection (FCOS)

During CCN1, FCOS was selected over Faster R-CNN. On a 640×480 input the float FCOS reached an IoU of 0.874 and an mAP at IoU > 0.75 of 0.956, against 0.853 and 0.901 for Faster R-CNN, at 45.78 FPS against 37.66 FPS on the RTX 3070, with 32.1 M parameters against 43 M. Additionally, it was not possible to train Faster R-CNN with Vitis-AI QAT, so it was dropped from further consideration.

We then lowered the resolution to 384×288 , because we encountered memory issues when attempting QAT at the full 640×480 resolution. The best float model at that size was `licit-weal` (IoU 0.925). Quantization-aware training from that checkpoint produced `quare-delf` (IoU 0.858). IoU therefore fell by 0.067.

Table 6.1 FCOS, 384×288 . FLOAT vs QAT performance metrics from [AD-1].

	licit-weal (FLOAT)	quare-delf (QAT)	difference
Started from	COCO	licit-weal	—
Input [W × H]	384 × 288	384 × 288	—
IoU	0.925	0.858	-0.067
mAP	0.858	0.693	-0.165
mAP at IoU > 0.50	0.989	0.978	-0.011
mAP at IoU > 0.75	0.946	0.850	-0.096

6.2. Keypoint regression (MobilePose)

The float model selected in CCN1 was **blest-harl**, initialised from the DIOSSA-1 MobilePose checkpoint and trained on 20 keypoints. Raising the input from 224 × 224 to 448 × 448 did not improve accuracy, although a further study is planned [AD-2] to determine whether the network would benefit from a larger input image size, specifically at closer distances.

The QAT model was **bijou-rasp**, started from **blest-harl**. Its L2 pixel error was 5.303 px on the camera image. The increase in L2 error due to quantization was 1.504 px.

Table 6.2 MobilePose, 224 × 224 input, 20 keypoints. FLOAT vs QAT performance metrics from [AD-1].

	blest-harl (FLOAT)	bijou-rasp (QAT)	difference
Started from	DIOSSA-1 MobilePose	blest-harl	—
Input	224 × 224	224 × 224	—

	blest-har1 (FLOAT)	bijou-rasp (QAT)	difference
Keypoints	20	20	—
L2 on the camera image [2590 × 1942], px	3.799	5.303	-1.504
L2 on the network input [224 × 224], px	1.114	1.238	-0.124

7. Latency

Both platforms run the same split: the convolutional edge on the accelerator, and the tail on the CPU through ONNX Runtime. The times below are those two pieces, measured separately and then added. Image decode and the pose solver are not included.

7.1. Xilinx

The Xilinx numbers come from the earlier benchmarking campaign on the Xilinx MPSoC that addressed the RID DS-10. The DPU time was measured with `xdputil benchmark` running for 60 seconds. The CPU time was measured with `onnxruntime_perf_test` running for 20 seconds, and the thread count of that ONNX run was not pinned.

Table 7.1 Xilinx MPSoC. DPU time and ONNX Runtime head, measured separately.

	DPU	CPU head	Sum
MobilePose 224 × 224	2.95 ms (338.5 FPS)	56.84 ms	59.8 ms (16.7 FPS)
FCOS 384 × 288	51.81 ms (19.30 FPS)	4.48 ms	56.3 ms (17.8 FPS)

On the DPU, MobilePose is a small fraction of the processing chain. Almost all of the summed time is spent in the CPU tail. FCOS is the opposite: the DPU is the slow part, and the CPU tail is under 5 ms.

7.2. PolarFire

PolarFire times were measured on the devkit board running VectorBlox SDK 3.1. The edge parts of the models were compiled onto the VectorBlox core with the V1000 configuration and no compression. Each network was run on 50 images from the CCN1 sample, one loop per image. The standard deviation of the accelerator time is 0.01 ms for MobilePose and 0.02 ms for FCOS, showing that the measurements are stable.

Table 7.2 PolarFire VectorBlox. Mean over 50 images.

Metric	MobilePose 224 × 224	FCOS 384 × 288
Accelerator	17.20 ms (58.2 FPS)	394.67 ms (2.53 FPS)
Dequantize	4.33 ms	1.81 ms
ONNX Runtime	16.80 ms	13.37 ms
End-to-end	38.33 ms (26.1 FPS)	409.85 ms (2.44 FPS)

7.3. Comparison

Table 7.3 Accelerator time and end-to-end time.

Metric	Xilinx DPU	PolarFire VectorBlox
MobilePose accelerator	2.95 ms	17.20 ms
MobilePose end-to-end	59.8 ms (sum)	38.33 ms
FCOS accelerator	51.81 ms	394.67 ms

Metric	Xilinx DPU	PolarFire VectorBlox
FCOS end-to-end	56.3 ms (sum)	409.85 ms
Full pose estimation chain	$59.8 + 56.3 = 116.1$ ms	$38.33 + 409.85 = 448.18$ ms

MobilePose on the VectorBlox core is slower than on the DPU (17.2 ms against 2.95 ms). The ONNX head on the PolarFire RISC-V cores is faster than the head measured on the MPSoC (21.1 ms against 56.8 ms). Added together, the PolarFire chain is the shorter one: 38.3 ms against 59.8 ms.

FCOS does not follow that pattern. The VectorBlox core takes 395 ms, about 7.6 times the DPU, and the head does not compensate. The end-to-end time is 410 ms, 2.4 frames per second, against 56 ms on the Xilinx accelerator.

Therefore, the latency requirement (1 Hz) is met when switching from Xilinx to PolarFire.

Improving the processing time of the FCOS model on PolarFire and exploring a smaller object detection model are both potential areas for future work, as the current model exhibits excessive complexity relative to the application's requirements.

8. Qualitative overlays

The figures below are PolarFire outputs from the CCN1 test set, drawn on the network input images (224×224 for MobilePose, 384×288 for FCOS). Ground truth is shown in green and predictions in red. Yellow lines show ground-truth and prediction keypoint pairs.

8.1. Keypoint regression

For some samples, predictions line up accurately with the ground truth, but for others they do not. These errors are quite large for the outliers, and therefore we cannot provide reliable accuracy metrics (L2 pixel error, etc.) for this model.

On these four frames the red keypoints sit on the same structure as the green ones (body, antennas, and nozzle), but some keypoints are swapped with others. This must be studied in more detail to determine whether the cause is the model or the quantization process.

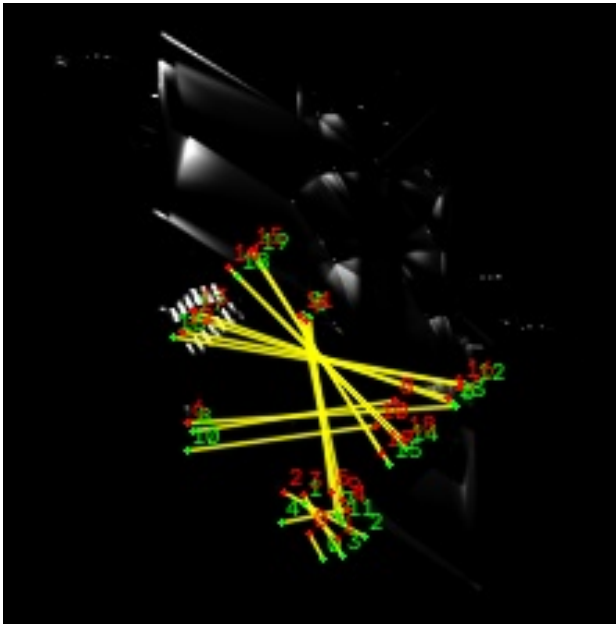


Fig. 8.1 MobilePose, `VisCam_0001775` .

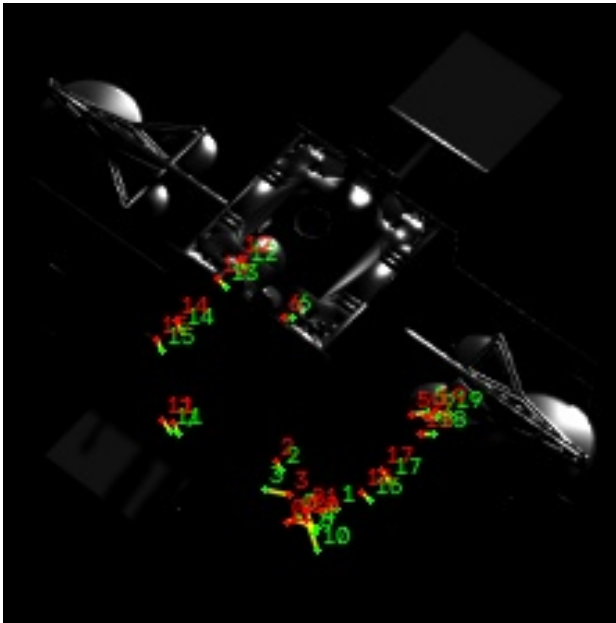


Fig. 8.2 MobilePose, `VisCam_0012391` .

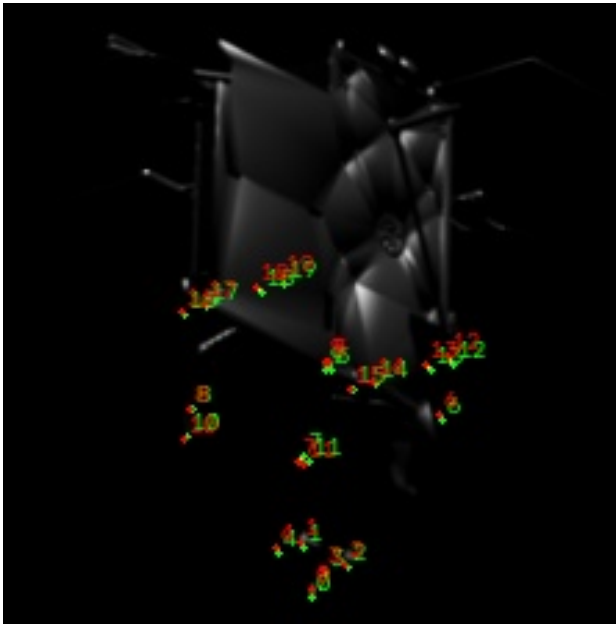


Fig. 8.3 MobilePose, `VisCam_0005108` .

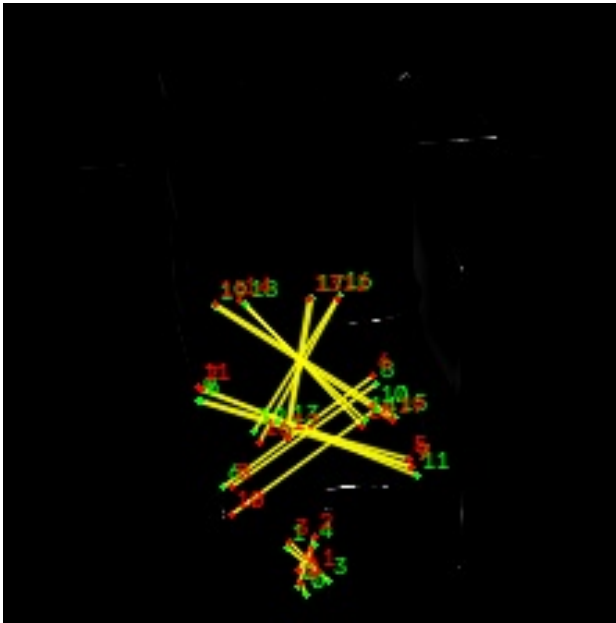


Fig. 8.4 MobilePose, `VisCam_0029974` .

8.2. Object detection

On these frames the network returns a detection, and the bounding box falls inside the spacecraft. The box is much smaller than the green ground-truth extent. That is visible in the samples below. This behaviour can be observed consistently in most of the samples, and therefore we cannot provide reliable accuracy metrics (IoU, mAP, etc.) for this model. This problem can be caused by the quantization process, or by faulty dequantization and further decoding of the output on the CPU.

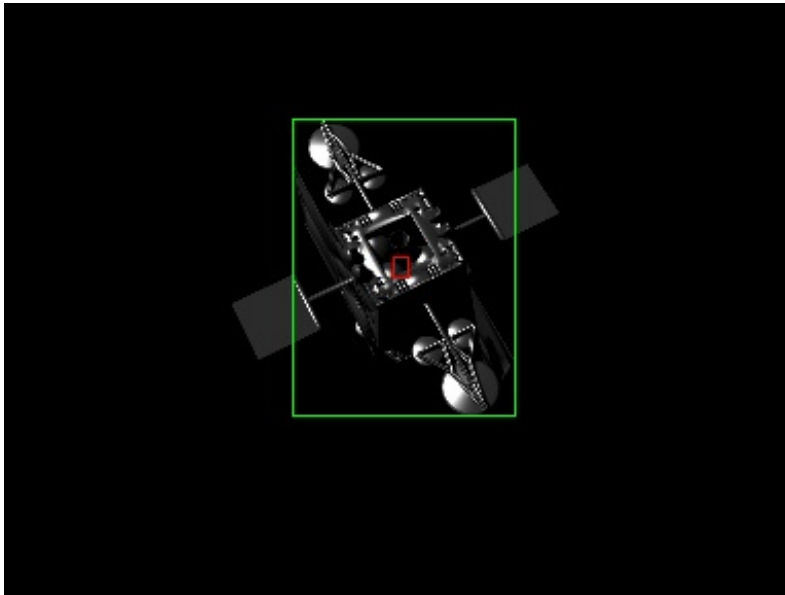


Fig. 8.5 FCOS, [VisCam_0002932](#) .

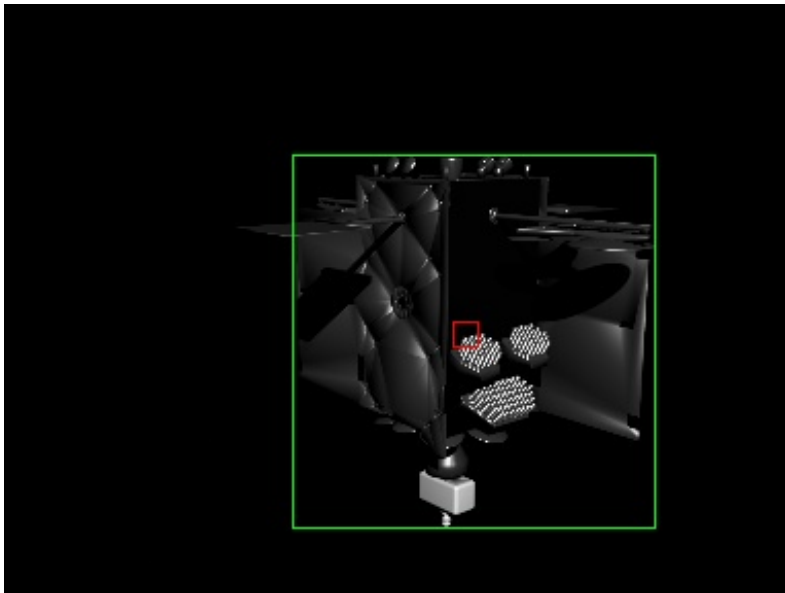


Fig. 8.6 FCOS, [VisCam_0008192](#) .

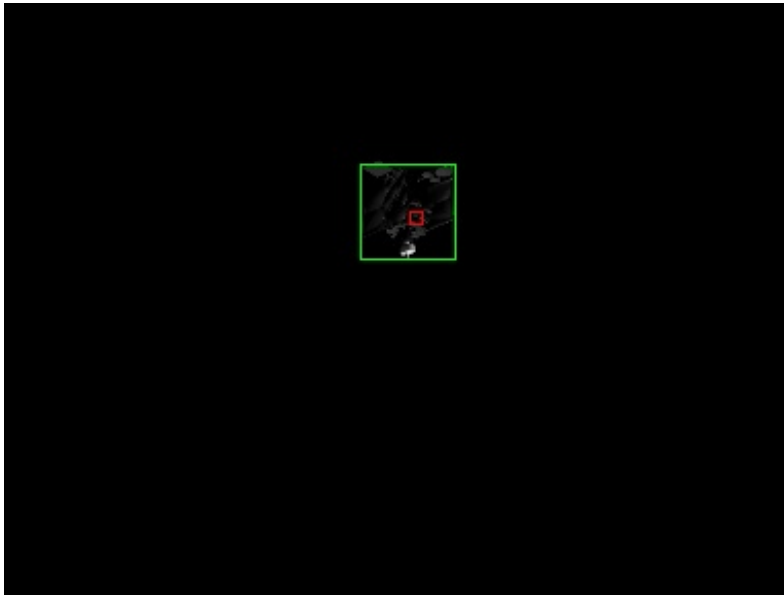


Fig. 8.7 FCOS, `VisCam_0031435` .

9. Conclusion

The QAT checkpoints from the DIOSSA-CCN1 project were compiled for VectorBlox and run on the PolarFire SoC, with the convolutional edge on the accelerator and the tail on the CPU using ONNX Runtime.

On latency, MobilePose end-to-end is shorter on PolarFire than the summed Xilinx measurement (38 ms against 60 ms). FCOS is not: the VectorBlox core takes about 7.6 times the DPU time, and the chain runs at 2.4 frames per second. The latency requirement (1 Hz) is met when switching from Xilinx to PolarFire.

On the qualitative overlays, MobilePose keypoints follow the ground truth on some of the frames shown above and not on others. FCOS places a box on the spacecraft, and that box does not cover the ground-truth extent.

10. Open items

1. Even though the models went through the embedding process, we did not validate their accuracy on the CCN1 test set, and therefore could not compare whether there is a performance difference between Xilinx and PolarFire. Additional effort must be put into verifying the underlying reason for the large performance degradation. One possible reason is the quantization process. Another is that the weights reused from CCN1 are not optimal for

quantization on PolarFire specifically. We might have to look into running QAT tailored for the PolarFire platform.

2. Consider a smaller object detection model that maintains accuracy when quantized. FCOS is quite a large model, and it is not clear whether it is necessary for the application. Detecting a single spacecraft is arguably simpler than keypoint regression, so a smaller model should be sufficient. For example, models from the YOLO family are widely used for object detection on embedded devices.
3. Verify that both models can be run at the same time on the PolarFire platform. So far, each model has been run one after the other. We shall investigate whether they can be run concurrently.

